

Java

Exception handling, multi threading

```
class Excpt{

    static void checkAge(int age){
        if(age < 18){
            throw new ArithmeticException("Access denied");
        }
    }
}

public class App {

    public static void main(String[] args) throws Exception {

        // Excpt obj = new Excpt();

        // obj.checkAge(20);

        // Excpt.checkAge(2);

        // exception => arithmetic, ....

        try {
            String s = "hello";

            int num = Integer.parseInt(s);

            System.out.println(num);
            int arr[] = new int[5];

            arr[10] = 50;

            int result = 10/0;

        } catch(ArithmeticException obj) {
            System.out.println(obj.getMessage());
        } catch(ArrayIndexOutOfBoundsException e){
            System.out.println(e.getMessage());
        } catch(NumberFormatException obj){
            System.out.println("Invalid number");
        } catch(Exception e){
            System.out.println(e.getMessage());
        } finally{
            System.out.println("exception handling passed");
        }
    }
}
```

```

    }

    System.out.println("End of the program ");

    // check exception and unchecked exceptions

    // checked exception => checked at compile time
    // examples => IOException, SQLException

    // unchecked exception => checked at runtime
    // occur due to programming errors ,

    // examples => ArithmeticException ,
    // ArrayIndexOutOfBoundsException
}
}

```

```

class MyRunnable implements Runnable{
    public void run(){
        System.out.println("Runnable is running");
    }
}

class MyThread extends Thread{

    public void run(){
        System.out.println("Thread is running");
    }
}

public class Multithreading {
    public static void main(String[] args) throws InterruptedException {

        System.out.println(Thread.MIN_PRIORITY);
        System.out.println(Thread.MAX_PRIORITY);
        System.out.println(Thread.NORM_PRIORITY);
        MyRunnable obj1 = new MyRunnable();

        Thread t2 = new Thread(obj1);

        MyThread t1 = new MyThread();
    }
}

```

```
t2.start();
t1.start();

// Thread.sleep(10000);

System.out.println(t2.isAlive());

t1.join();
t2.join();

t1.setName("MyThread");
t2.setName("MyRunnable");

System.out.println(t2.getName());
System.out.println(t1.getName());

t2.setPriority(7);
System.out.println(t2.getPriority());

System.out.println("Hello there");

System.out.println("End of the program");


// it is one of the ways to achieve multi-tasking

// Thread => it is the light weight sub process

// threads shares the common memory area


// thread lifecycle

// new,
// runnable
// running,
// non-runnable,
// terminated

// createing thread => Extending Thread class


}

}
```

```
class Counter{
    int count =0;

    synchronized void increment(){
        count++;
    }
}

public class Synchronized {

    public static void main(String[] args) throws InterruptedException {
        System.out.println("Multithreading synchronized");

        Counter obj = new Counter();

        // obj.increment();
        // obj.increment();

        // lambda expression
        Thread t1 = new Thread( ()->{

            for(int i=1; i<=20000; i++){
                obj.increment();
            }
        } );

        Thread t2 = new Thread( ()->{

            for(int i=1; i<=30000; i++){
                obj.increment();
            }
        } );

        t1.start();
        t2.start();

        t1.join();
        t2.join();

        System.out.println(obj.count);

    }
}
```

Theory

1. Exception Handling

Definition: An exception is an event that disrupts the normal flow of a program. Use case: Helps handle runtime errors gracefully without crashing the program. Key point:

- Checked exceptions → verified at compile time (e.g., IOException, SQLException).
- Unchecked exceptions → occur at runtime (e.g., ArithmeticException, ArrayIndexOutOfBoundsException).

Syntax:

```
try {  
    int result = 10/0;  
} catch(ArithmeticException e) {  
    System.out.println("Error: " + e.getMessage());  
} finally {  
    System.out.println("Cleanup code");  
}
```

2. Multithreading

Definition: Multithreading is the process of executing multiple threads concurrently. Use case: Achieves multitasking, improves performance. Key point:

- Thread creation → extend Thread class or implement Runnable interface.
- Thread lifecycle → New → Runnable → Running → Non-runnable → Terminated.

Syntax:

```
class MyThread extends Thread {  
    public void run() { System.out.println("Thread running"); }  
}
```

3. Synchronization

Definition: Synchronization ensures that only one thread accesses a shared resource at a time. Use case: Prevents race conditions in multithreaded programs. Key point: Achieved using synchronized keyword.

Syntax:

```
class Counter {  
    int count = 0;  
    synchronized void increment() { count++; }  
}
```

Coding Round Questions

- Exception handling → Write a program to handle `ArithmeticException`, `ArrayIndexOutOfBoundsException`, and `NumberFormatException`.
- Checked vs Unchecked → Explain difference between checked and unchecked exceptions with examples.
- Thread creation → Demonstrate thread creation using both `Thread` and `Runnable`.
- Thread lifecycle → Explain thread lifecycle stages with a diagram.
- Synchronization → Write a program to increment a counter using two threads with synchronization.
- Thread priority → Show how to set and get thread priority.

⚡ Machine Coding Round Challenge

Problem: Bank Account System

Requirements:

- Create a `BankAccount` class with balance.
- Methods:
 - `deposit(amount)` → adds money.
 - `withdraw(amount)` → subtracts money.
- Use exception handling to prevent withdrawing more than balance.
- Use multithreading to simulate multiple users depositing/withdrawing simultaneously.
- Use synchronization to ensure thread-safe operations.